

Solution Requirements

Date	July 9, 2026
Team ID	SWTID-2026-1262
Project Name	OptiCrop: Smart Agricultural Production Optimization Engine
Maximum Marks	4 Marks

Define Functional and Non-Functional Requirements:

Create a comprehensive list of requirements to define exactly what your solution will do and how well it will perform.

- **Functional Requirements (FR)** define specific behaviors, functions, or features of the system (What the system should *do*).
- **Non-Functional Requirements (NFR)** specify the criteria that judge the operation of a system, rather than specific behaviors (How the system should *be*).

Fill out the descriptions and necessary details for each category provided below.

Step 1: Functional Requirements (FR):

S.No	Requirement Category	Requirement Description	Priority (High/Medium/Low)
1	Authentication	For the local deployment scope, the Flask web console executes a direct session handshake; a persistent state is securely verified at app launch to confirm if the machine learning model.pkl binary is loaded and present on the server.	Medium
2	Authorization levels	The system maps data boundaries to the 3 core scenarios: Farmers get localized crop recommendations; Researchers access the	High

		multi-model validation matrix; Policymakers view dynamic agro-ecological zoning profiles	
3	External interfaces	The system interfaces directly with local disk storage paths via standard Python file utilities to load the persisted binary pipeline (model.pkl).	High
4	Transactions processing	The web core intercepts incoming POST form data vectors. It parses inputs (N, P, K, temperature, humidity, ph, rainfall), formats them into a clean NumPy multidimensional array (np.array), and maps it instantly through the model matching engine.	High
5	Reporting	The engine renders dynamic, highly contextual summary blocks via result.html. It displays calculated matching metrics, crop mismatch alerts (Scenario 2), and detailed resource efficiency roadmaps (Scenario 3) based on input chemistry vectors	High
6	Business rules, etc.	Strict Range-Boundary Rules: The Flask backend rejects fields falling outside explicit bio-chemical constraints: N (0–150 mg/kg), P (0–100 mg/kg), K (0–220 mg/kg), temp (10–50°C), humidity (0–100%), pH (4–9), and rainfall (0–400 mm).	High

7	Compliance to laws or regulations	All telemetry data streams run locally within the application bounds. The data profiling boundaries ensure total alignment with baseline data safety principles, as no personal profiling records are retained or tracked externally.	Medium
8	Other	Automated algorithmic backup evaluation fallback: The train_model.py module evaluates KNN, Decision Tree, Random Forest, and Logistic Regression models, dynamically verifying model performance before generating the pickled snapshot.	Medium

Step 2: Non-Functional Requirements (NFR):

S.No	NFR Category	Requirement Description	Target Metric / Acceptance Criteria
1	Performance & Speed	The time required for the Flask server route to validate the numeric web form array and output the machine learning inference result from the loaded pickle matrix.	Processing turnaround and HTML dashboard rendering completes in < 1.5 seconds on target hardware profiles.

2	Scalability	The engine's capacity to handle standard concurrent testing scenarios and execute algorithmic model processing arrays across isolated tabs.	Supports up to 50 concurrent model requests seamlessly inside a local development environment.
3	Security & Data Privacy	Secure boundary protection, safe data type mapping via the native Flask validation framework, and memory safety configurations during the file loading stage.	Zero vulnerability metrics on input parameters; text injection or string overrides are completely blocked by float type conversions.
4	Reliability & Availability	Structural system uptime, fault-tolerant execution structures, and localized exception catching logic during unexpected value entries.	The localized application maintains 99.9% script execution availability with zero runtime crashes due to unhandled numerical exceptions.
5	Usability & Accessibility	Responsive layout hierarchy enabling intuitive navigation through your application workflows via different smartphone, tablet, or web browser formats.	Core layouts match the visual cleanliness of WCAG 2.1 AA design lines , ensuring scannable parameters across the Bootstrap interface tabs.
6	Other	Comprehensive portability configuration across diverse machine setups and platforms using clean package setups.	Runs natively on any x86/ARM hardware running Python 3.10+ via standard pip configuration packages.